

BreachPoint CTF — Quality Assurance & Testing Report

Formal QA Test Cases, Defect Tracking Matrix & Pre-CTF System Validation

Document ID / Author:	BP-QA-REP-V2.1 Haggalla H.H.D.S (IT24100975) - Member 4
Module / Course:	IE3132 - Penetration Testing & Ethical Hacking (Assignment 02)
Target Platform:	BreachPoint CTF (6 Stages, 1,550 Total Points)
Host & Port Architecture:	Machine 1: Port 80 (Nginx Reverse Proxy) & Port 2222 (SSH) Machine 2: Port 8000 (CTFd)
Final Test Status:	100% PASSING — Verified Against Production Containers & Source Code

1. Executive QA Summary & Scope

As Member 4, the primary responsibilities are end-to-end integration, flaw identification, defect fixing, unintended shortcut prevention, and comprehensive documentation.

This report documents the verification of the 16-item Pre-CTF System Validation Checklist originally committed in Assignment 01, the formal test cases (TC-01 through TC-08), the bug tracking matrix (BUG-01 through BUG-05), and the execution of the automated end-to-end LO3 validation script (validate_killchain_e2e.py).

Key Testing Metrics:

- Total Test Cases Executed: 8 (TC-01 through TC-08)
- Defects Identified: 5 (BUG-01 through BUG-05)
- Defects Resolved & Retested: 5 (100% resolution)
- Unintended Solution Shortcuts Prevented: 4
- Automated Validation Status: PASS (Cryptographic baseline verified via validate_killchain_e2e.py)

2. Actual Network Topology & Stage Mapping

All web stages are containerized and routed exclusively through an Nginx Reverse Proxy on Port 80, while lateral movement transitions to Port 2222 for the Linux environment:

Component	Port / Access Path	Container Name	Target Function & Exploitation Flow
Nginx Reverse Proxy	80/TCP (Public)	nginx-proxy	Routes path-based HTTP requests (/ , /api/ , /novatech-portal/ , /files/) to internal challenge containers.
Stage 1: Open Eyes	80/TCP (via /)	s1-web	Corporate portal (about.html exposes Flag Part 1; novatech_logo.png EXIF metadata exposes Flag Part 2).
Stage 2: Knock Knock	80/TCP (via /api/)	s2-api	Hidden REST API endpoint discovered via Gobuster;

leaks credentials and Stage 2 flag.

Stage 3: The Login Wall	80/TCP (via /novatech-portal/)	s3-portal	Employee portal backed by MySQL (s3-db); client-side cookie nova_role=guest manipulated to admin.
Stage 4: Scrambled Secrets	80/TCP (via /files/)	s4-files	Network diagnostic ping tool vulnerable to command injection; exfiltrates /opt/backups/config_backup.enc.
Stage 5: Dead Logs	2222/TCP (SSH)	s5s6-ssh	Encrypted archive /tmp/suspicious.zip cracked with John the Ripper (shadow123) to extract notes.txt.
Stage 6: Root of Evil	2222/TCP (SSH)	s5s6-ssh	Vulnerable SUID binary /usr/local/bin/reader executing less; GTFOBins breakout to root for /root/flag.txt.

3. Defect Tracking Matrix (Bugs Found & Fixed)

Bug ID	Stage	Defect Description	Severity	Fix Implemented	Status
BUG-01	Stage 1	Entire flag was exposed in a single visible HTML comment on index.html.	High	Split flag into two parts: Part 1 in about.html protected by JS contextmenu/Ctrl+U blockers; Part 2 embedded in novatech_logo.png EXIF metadata.	VERIFIED CLOSED
BUG-02	Stage 2	API endpoint ran on an unrouted port with zero architecture integration.	Medium	Configured nginx-proxy/nginx.conf to route location /api/ internally to s2-api, requiring Gobuster / dirb directory fuzzing.	VERIFIED CLOSED
BUG-03	Stage 3	Valid credentials immediately displayed flag with zero role verification.	Critical	Modified login.php to issue a low-privilege nova_role=guest cookie; dashboard.php now strictly requires cookie elevation to nova_role=admin.	VERIFIED CLOSED
BUG-04	Stage 4	config_backup.enc located inside web root (/var/www/html/backups/), allowing direct download bypass.	High	In Dockerfile, moved backup to /opt/backups/ and deleted /var/www/html/backups/. Forces command injection: 127.0.0.1; cat /opt/backups/config_backup.enc.	VERIFIED CLOSED
BUG-05	Stage 5	Cleartext key /home/deploy/.secret_key left in container build, bypassing password cracking.	Critical	Commented out .secret_key creation in setup.sh; enforced legitimate offline dictionary cracking using zip2john and John the Ripper (rockyou.txt).	VERIFIED CLOSED

4. Formal Test Cases & Verification Results (TC-01 through TC-08)

TC-01: Stage 1 — Source Inspection Barrier & HTML Clue Extraction

Target Stage & Focus:	Stage 1 (Open Eyes - Reconnaissance / OSINT)
Test Objective:	Verify that direct browser inspection shortcuts (right-click, Ctrl+U) are intercepted and Part 1 of the flag is retrieved via developer tools or curl.
Initial Finding (Pre-Fix):	FAIL (BUG-01) — Entire flag was dumped into a single visible HTML comment on index.html.
Test Execution Steps:	1. Navigate to <code>http://<IP>:80/about.html</code> 2. Attempt right-click inspect -> JavaScript alert triggers; context menu blocked 3. Attempt Ctrl+U -> Keypress intercepted 4. Execute <code>curl -s http://<IP>:80/about.html grep -i 'BPCTF'</code> 5. Extract Part 1: <code>BPCTF{r3c0n_</code>
Expected Outcome:	Client-side shortcuts intercepted. HTTP response yields Part 1 of flag: <code>BPCTF{r3c0n_</code> .
Unintended Shortcut Check:	Prevents accidental zero-effort solves via simple right-click.

Result Status: **PASS (Defect BUG-01 Closed)**

TC-02: Stage 1 — Image Metadata (EXIF) Clue & Flag Assembly

Target Stage & Focus:	Stage 1 (Open Eyes - Steganography)
Test Objective:	Verify that <code>novatech_logo.png</code> contains Part 2 of the flag in its metadata and assembles to <code>BPCTF{r3c0n_m4st3r}</code> .
Initial Finding (Pre-Fix):	PASS — EXIF metadata preserved across container build.
Test Execution Steps:	1. Download logo: <code>curl -s http://<IP>:80/novatech_logo.png -o logo.png</code> 2. Inspect metadata: <code>exiftool logo.png grep -i 'Artist'</code> 3. Observe metadata tag: <code>Artist: m4st3r}</code> 4. Assemble Part 1 and Part 2: <code>BPCTF{r3c0n_m4st3r}</code>
Expected Outcome:	Metadata extraction cleanly yields <code>'m4st3r'</code> . Assembled flag matches CTFd baseline.
Unintended Shortcut Check:	Ensures participants must perform actual forensic metadata analysis.

Result Status: **PASS**

TC-03: Stage 2 — Reverse Proxy API Path Fuzzing & Unauthenticated Disclosure

Target Stage & Focus:	Stage 2 (Knock Knock - Web Recon / API Security)
Test Objective:	Ensure <code>/api/</code> endpoint is discoverable via Gobuster over port 80 and exposes admin credentials and Flag 2.
Initial Finding (Pre-Fix):	FAIL (BUG-02) — API ran on an unmapped standalone port with no reverse proxy integration.

Test Execution Steps:	1. Run directory enumeration: <code>gobuster dir -u http://<IP>:80 -w /usr/share/wordlists/dirb/common.txt</code> 2. Gobuster discovers: <code>/api/</code> (Status 301/200) 3. Query endpoint: <code>curl -s http://<IP>:80/api/v1/users/</code> 4. Inspect JSON payload for users, credentials, and Stage 2 flag
Expected Outcome:	JSON response returns admin credentials (admin : N0v4T3ch@dm1n) and Stage 2 flag: BPCTF{4p1_3xp0sur3_l34ks_cr3ds}.
Unintended Shortcut Check:	Reverse proxy ensures uniform single-port (80) penetration testing.

Result Status: PASS (Defect BUG-02 Closed)

TC-04: Stage 3 — RBAC Cookie Manipulation & Privilege Escalation

Target Stage & Focus:	Stage 3 (The Login Wall - Access Control / OWASP A01)
Test Objective:	Confirm that authenticating with Stage 2 credentials issues a low-privilege nova_role=guest cookie, and altering it to admin unlocks Flag 3.
Initial Finding (Pre-Fix):	FAIL (BUG-03) — Submitting valid credentials directly dumped the flag without access control.
Test Execution Steps:	1. POST to <code>http://<IP>:80/novatech-portal/login.php</code> with <code>admin:N0v4T3ch@dm1n</code> 2. Verify Set-Cookie header: <code>nova_role=guest; path=/</code> 3. Request <code>dashboard.php</code> with <code>nova_role=guest</code> -> Displays 'Access Denied: Standard User' 4. Tamper cookie: <code>nova_role=admin</code> 5. Request <code>dashboard.php</code> with modified cookie
Expected Outcome:	Dashboard unlocks elevated card: Flag 3 (BPCTF{4uth_p0rt4l_4cc3ss}).
Unintended Shortcut Check:	Prevents bypass of authorization testing.

Result Status: PASS (Defect BUG-03 Closed)

TC-05: Stage 4 — Web Root Isolation & Direct Download Prevention

Target Stage & Focus:	Stage 4 (Scrambled Secrets - Unintended Path Check)
Test Objective:	Verify that <code>config_backup.enc</code> cannot be directly fetched via HTTP GET request over <code>/files/</code> .
Initial Finding (Pre-Fix):	FAIL (BUG-04) — Backup was stored in <code>/var/www/html/backups/</code> and accessible via direct download.
Test Execution Steps:	1. <code>curl -I http://<IP>:80/files/backups/config_backup.enc</code> 2. <code>curl -I http://<IP>:80/files/config_backup.enc</code> 3. <code>curl -I http://<IP>:80/files/opt/backups/config_backup.enc</code>
Expected Outcome:	All requests return HTTP 404 Not Found. File is completely isolated outside web root.
Unintended Shortcut Check:	Forces participants to exploit the command injection vulnerability.

Result Status: PASS (Defect BUG-04 Closed)

TC-06: Stage 4 — Command Injection Execution & Cryptographic Decoding

Target Stage & Focus:	Stage 4 (Scrambled Secrets - Web Exploitation & Crypto)
Test Objective:	Verify command injection via ping input: 127.0.0.1; cat /opt/backups/config_backup.enc and decode the recovered ciphertext.
Initial Finding (Pre-Fix):	PASS — Command injection parameter correctly accepts shell delimiters.
Test Execution Steps:	1. Access http://<IP>:80/files/ 2. Submit parameter: ?host=127.0.0.1; cat /opt/backups/config_backup.enc 3. Capture base64 ciphertext from HTML response 4. Base64 decode to reveal ROT13 ciphertext 5. Apply ROT13 rotation to reveal deploy SSH password and Flag 4
Expected Outcome:	Recovers deploy user password (D3pl0y#S3cur3!) and Flag 4: BPCTF{crypt0_l4y3rs_d3c0d3d}.
Unintended Shortcut Check:	Restricted ping packet count (ping -c 2) prevents server timeout.

Result Status: **PASS**

TC-07: Stage 5 — Forensic Zip Extraction & Password Cracking Integrity

Target Stage & Focus:	Stage 5 (Dead Logs Tell Tales - Digital Forensics)
Test Objective:	Verify that /tmp/suspicious.zip requires John the Ripper cracking and cannot be bypassed via cleartext keys.
Initial Finding (Pre-Fix):	FAIL (BUG-05) — Debugging key /home/deploy/.secret_key allowed bypassing zip cracking.
Test Execution Steps:	1. SSH to target: ssh deploy@<IP> -p 2222 with password D3pl0y#S3cur3! 2. Check for cleartext keys: ls -la /home/deploy/.secret_key (Not found) 3. Download /tmp/suspicious.zip to Kali attack box 4. Generate hash: zip2john suspicious.zip > zip.hash 5. Crack hash: john --wordlist=/usr/share/wordlists/rockyou.txt zip.hash 6. Unzip with cracked password (shadow123) and inspect notes.txt
Expected Outcome:	Password cracked successfully. notes.txt reveals Flag 5 (BPCTF{f0r3ns1cs_4rt3f4ct_r3c0v3r3d}) and hints at SUID reader binary.
Unintended Shortcut Check:	Eliminated cleartext key bypass, enforcing legitimate forensic cryptanalysis.

Result Status: **PASS (Defect BUG-05 Closed)**

TC-08: LO3 End-to-End Cryptographic Kill-Chain Validation

Target Stage & Focus:	All Stages (Integration & Regression Testing)
Test Objective:	Validate SHA-256 integrity of all 6 flags and confirm automated grading engine rejects invalid flags.
Initial Finding (Pre-Fix):	PASS — Verified across regression suite.

Test Execution Steps:	1. Run validation engine: <code>python scripts/validate_killchain_e2e.py</code> 2. Verify SHA-256 baseline matches for Stages 1 through 6 3. Execute anti-forgery check: Submit malformed flag <code>BPCTF{wrong_flag_test}</code>
Expected Outcome:	All 6 flags match SHA-256 baseline signatures (100% integrity). Malformed flag rejected [PASS]. Total score: 1,550 points.
Unintended Shortcut Check:	Guarantees absolute score integrity and zero flag collisions.

Result Status: PASS

5. Verification of Assignment 01 Checklist (16 Items)

Every item from Section 11.1 of the approved Assignment 01 specification was re-tested and verified prior to opening participant access. This table confirms 100% compliance across all 16 items:

Item #	Verification Item	Execution Method	Expected / Actual Result	Result
1	All containers start cleanly	Run <code>docker compose up -d</code> on both machines, then verify Up status	All containers running healthy with zero build/runtime errors	PASS
2	Nginx routing (Machine 1)	Access <code>http://<IP>/, /api/v1/users, /novatech-portal, and /files/</code>	Each URL path routes accurately to designated challenge container	PASS
3	SSH access (S5/S6)	Execute <code>ssh deploy@<Machine1 IP> -p 2222</code> with Stage 4 credentials	Successful terminal login as deploy user	PASS
4	CTFd loads all challenges	Inspect CTFd admin portal; verify all 6 challenges and point allocations	All 6 stages active with correct 1,550 point progression	PASS
5	S1 flag retrieval	View source of <code>about.html</code> and inspect <code>novatech_logo.png</code> with <code>exiftool</code>	Both flag halves retrieved; assembled flag <code>BPCTF{r3c0n_m4st3r}</code>	PASS
6	S2 API endpoint	Execute <code>gobuster</code> dir scan on port 80; access <code>/api/v1/users/</code>	Returns JSON payload with admin credentials and Stage 2 flag	PASS
7	S3 portal login	Navigate to <code>/novatech-portal</code> , authenticate with gathered admin credentials	Dashboard renders; <code>role=guest</code> issued; cookie tamper unlocks flag	PASS
8	S3 invalid login rejection	Submit deliberately invalid credentials into portal login form	Authentication rejected with error banner; zero flag/data exposure	PASS
9	S4 decode chain	Inject <code>127.0.0.1; cat /opt/backups/config_backup.enc</code> on <code>/files/</code>	Encrypted payload exfiltrated; decoded (Base64+ROT13) for flag & SSH key	PASS

10	S5 forensic chain	SSH in, transfer /tmp/suspicious.zip, crack via zip2john + John the Ripper	Password shadow123 cracked; notes.txt reveals Flag 5 & SUID hint	PASS
11	S6 SUID exploit	Enumerate SUID permissions, execute /usr/local/bin/reader, break out via less	Root shell obtained (!/bin/sh); /root/flag.txt extracted	PASS
12	All flags validate in CTFd	Validate each flag string against CTFd SHA-256 baseline specifications	All 6 flags accepted by CTFd automated grading engine	PASS
13	Wrong flag rejection	Submit deliberately malformed flag BPCTF{wrong_flag_test}	Malformed flag strictly rejected; false positives prevented	PASS
14	Inter-container isolation	Internal bridge ctf-challenges with internal:true blocks external egress	Inter-container isolation verified; challenge integrity protected	PASS
15	Machine isolation	Attempt unauthorized network breakouts from s5s6-ssh container	Machine 2 CTFd database and external networks isolated	PASS
16	Container reset	Execute docker compose down -v and docker compose up -d	Environment fully restores to pristine state; challenges repeatable	PASS